Smaller complete checkers for iPRA

Exact reductions and their limits

November 11, 2026

Result and qualification. A complete checker with validation of guarded public goals shrinks from 1,201 to 921 normalized Python bytes. A separate checker for the full native Eq/D public language shrinks from 803 to 788 bytes. A compact checker shrinks from 521 to 520 bytes, with an AST-size tradeoff. These are exact implementation refinements of inherited presentations. **No new arithmetic or logical axiom deletion was established.** The requested further mathematical simplification therefore remains open.

The comparison includes the three incoming archives present in the specified repository directory at commit

`8eb71c1e570e07ef45ead686628e77f18ef34798`.

The report supplies explicit rule inventories, preservation proofs, the scope of the bi-interpretations, reproducible source measurements, and the inherited arithmetic bridge. It distinguishes a smaller proof checker from a new mathematical axiomatization throughout.

Prepared in response to the request for a simpler theory equiconsistent and bi-interpretable with intuitionistic primitive recursive arithmetic. The included foundational appendices are attributed copies of earlier work in the supplied repository.

Contents

1 The result, the comparison, and the unmet part **1**

2 The complete inherited native axiomatization **1**
2.1 Numerical terms and formulas . . . 2
2.2 Every axiom and inference rule . . . 2
2.3 Derived logic and induction . . . 3

3 Why the theory is bi-interpretable with full iPRA **4**
3.1 An ordinary first-order realization . . . 4
3.2 The arithmetic interpretations . . . 4
3.3 Both composites and consistency . . . 5

4 The guarded public language of the smallest checker **5**
4.1 Public and auxiliary syntax . . . 5
4.2 Sound interpretation of auxiliary formulas . . . 5
4.3 Public completeness and the composites . . . 6

5 Removing duplicate validation in the guarded checker **6**
5.1 The representation and the original predicate . . . 7
5.2 The smaller shape predicate . . . 7
5.3 The existing opener supplies every omitted bound . . . 7

6 The two smaller local changes **8**
6.1 Full native public formulas: 803 to 788 bytes . . . 8
6.2 Compact guarded profile: 521 to 520 bytes . . . 8

7 Rejected shortcuts and remaining mathematical work **9**
7.1 Why context length did not remove freshness cost . . . 9
7.2 Other non-results . . . 9
7.3 Further research questions . . . 9

8 Artifacts, reproduction, and assurance **10**

Attribution and scope of the inherited appendices **11**

A Complementary introductions with one local assumption **11**
A.1 Complete native syntax and rule inventory . . . 11
A.2 Exact equivalence and the unused-witness construction . . . 12
A.3 Induction and numerical cases separate cleanly . . . 13
A.4 The entire ordinary logical interface . . . 14
A.5 Whole-domain realizations and both formula composites . . . 15

A.6 What the theorem does and does not claim 16

B Internal scalar predecessor bootstrap **16**
B.1 Elementary terms and two finite cycles 16
B.2 A ramp with a global scaling identity 17
B.3 A comparator from scalar iteration 18
B.4 Saturated iteration gives predecessor 19
B.5 Dependency boundary 20

C From literal-equation induction to full intuitionistic PRA **20**
C.1 The exact hypotheses 20
C.2 Constructor separation and two elementary tests 21
C.3 Four elementary uses of equation induction 22
C.4 An equation invariant that reflects actual equality 23
C.5 Compilation of all quantifier-free formulas 24
C.6 Recovery of induction and the full theory 25

D Pairing, primitive recursion, and internal numerical composites **26**
D.1 Pairing from a bounded square-root search 26
D.2 Compiling each primitive recursion 27
D.3 Why both numerical round trips are internal 27

E Arithmetic strength and both ordinary interpretation composites **27**
E.1 The arithmetic dependency order 28

1 The result, the comparison, and the unmet part

The requested target was a simpler axiomatization than the current `ConciseIPRA` proposal. The investigation produced a smaller *implementation of existing complete presentations*, but no new axiomatically simpler arithmetic basis. This distinction is material: the large reduction below is in public-goal validation, while the only new compact byte record is a one-byte Boolean-expression change. It would be misleading to describe either as a mathematical breakthrough.

The comparison is the supplied repository at the commit on the title page [1]. The current incoming packages already improve the figures summarized for the older filed packages. All three incoming ZIP files were inspected and checked against their Git blob hashes; their provenance is included in the archive. In particular, the current compact coordinate envelope is

$$(\text{tokens}, \text{AST nodes}, \text{normalized bytes}) = (256, 278, 521),$$

attained by *three different programs*, not one program of that size. The incoming `one_context` package already contains crossed one-context equality transport, and `kernel_refinement` already contains the left-occurrence refinement [2]. Neither is proposed again here.

Complete module and input contract	Tokens	AST	Bytes	Gzip-9
Current compact, token profile	256	283	594	346
Current compact, AST profile	260	278	602	344
Current compact, byte profile	263	300	521	327
New compact byte profile	263	303	520	329
Current guarded public-goal validator + kernel	489	560	1201	571
New guarded public-goal validator + kernel	407	459	921	507
Current full native Eq/D public-goal interface	359	414	803	463
New full native Eq/D public-goal interface	358	413	788	460

Table 1: Matched whole-module measurements. Gzip is informational. The three interfaces have different contracts and must not be compared as if they were interchangeable.

The guarded validated module saves 280 bytes, approximately 23.3%, as well as 82 tokens and 101 AST nodes. The native validated module saves 15 bytes, one token, and one AST node. The compact module saves one byte but adds three AST nodes and two gzip bytes. Thus the compact coordinate envelope becomes $(256, 278, 520)$ only in the coordinatewise sense. There is no claim of an absolute minimum or of improvement in all coordinates of the compact byte program.

Recommended use. For full native Eq/D public formulas, use `code/checker_native.py`. For the deterministic guarded public presentation, use `code/checker_guarded.py`. The 520-byte `code/checker_compact.py` retains the predecessor's canonical guarded-public-goal contract and does not include its validator. The two validated programs validate the *claimed public goal*; they do not impose a new exact integer/list grammar on every unused certificate field.

2 The complete inherited native axiomatization

This section states the mathematics being preserved, rather than counting an unchanged arithmetic theory as a new construction. It is convenient to use the inherited Choice-DI presentation [3], which has the same sequents as the Lean native kernel described in the repository.

2.1 Numerical terms and formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the state variable u is bound in h. The counter n and seed a are outside its scope. The update has no recursion-counter argument; other variables in it are fixed parameters. There is no numerical quantification over function descriptions and no uniform evaluator.

Formulas and the designated contradiction are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A, \qquad \bot_* := (0 = S0).$$

Bound variables are renamed before substitution. The ordinary readings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \longrightarrow \forall y\, B^\circ(y),$$
$$(\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \longrightarrow \forall y\, B^\circ(x, y)\big).$$

The Q constructor can be omitted from public syntax: an unused existential coordinate under D represents it. This omission is already part of the inherited construction, not a new reduction.

2.2 Every axiom and inference rule

The two nonlogical axiom schemas, available in every retained context, are

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

Contexts are finite reusable lists of formulas. The remaining primitive patterns of the Choice-DI presentation are the following:

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \text{Hyp}, \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B \quad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}\ \text{DE},$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}\ \text{DI}_{\text{case}},$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)}\ \text{DI}_{\text{prog}}.$$

Here F is any fixed native formula template, and E in the last rule must be a *literal equation*. In DE, b is fresh for Γ, the conclusion and the fixed parameters of the matrix. In either DI rule, z is fresh for Γ, the witness t, and the fixed matrix parameters. Displayed local assumptions are discharged.

These are eight patterns when Hyp and the two computation schemas are counted separately. Equality reflexivity, successor injectivity, ordinary logical rules, and quantifier-free induction are not further primitive axioms. Weakening, renaming, contraction, and cut are admissible. In the Lean

native kernel, Tr may instead be restricted to equation templates, and progressive introduction directly yields $\mathsf{Q}_y E(y)$; the repository proves equivalence of those presentations.

The measured native checker uses the inherited source restriction $s \not\equiv 0$ in Tr and the two introduction choices. Recovery of omitted transport instances is established in the included `reference/foundations/zero_source_admissibility.tex` and the repository's `NativeZeroSource` formalization. The source restriction is syntactic, not the assertion $s \neq 0$ inside arithmetic.

2.3 Derived logic and induction

Put $1 = S0$ and

$$C(v, a, b) := I(v, a; u.b), \qquad C(0, a, b) = a, \quad C(Sv, a, b) = b,$$

with unused u. Reflexivity follows from the computation $r = t$, where $r = I(0, t; u.0)$, by transporting $r = t$ along itself with template $v = t$. Symmetry and transitivity then follow by the usual equation templates.

Case DI with an unused witness, followed by DE and QE, gives numerical cases for every formula:

$$\frac{\Gamma \vdash A(0) \quad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}.$$

Progress DI with an unused witness similarly gives contextual equation induction:

$$\frac{\Gamma \vdash E(0) \quad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

The distinction between these two derived rules is essential.

There is a formula gate $G_v A$ that is true at $v = 0$ and equivalent to A at a successor. On equations it is

$$G_v(s = t) := C(v, 0, s) = C(v, 0, t),$$

and it commutes with the logical binders. A selector $M_v(A, B)$ is equivalent to A at zero and to B at a successor. It is built by aligning binder depths using the equivalence $A \dashv\vdash \mathsf{D}_{r,z} G_z A$, with unused r.

For $p(z) = I(z, 0; u.C(u, 1, 0))$, one may use the macros

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{r,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{r,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{r,z} G_z A(z). \end{aligned}$$

The newly bound coordinates are fresh. The first two fixed parity computations $p(1) = 1$, $p(2) = 0$ give conjunction projections. Cases and the selector give disjunction elimination and numerical coverage. The inherited appendix gives all introduction and elimination arguments, including both formula composites. Under $0 = 1$, selector computation and transport yield every equation, and case DI extends explosion to every native formula. No excluded middle for arbitrary quantified formulas is introduced.

3 Why the theory is bi-interpretable with full iPRA

3.1 An ordinary first-order realization

Elaborate each fixed iterator description into a function symbol. Given an earlier-stage update $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the equations (I0)–(IS). Let T_I be ordinary intuitionistic first-order equality logic over these symbols with those equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \longrightarrow \bot.$$

The last axiom is an explicit falsity bridge. A singleton structure would otherwise satisfy positive computation equations, so it must not silently be omitted from the ordinary realization.

The readings of Q and D in Section 2 give native soundness in T_I. Conversely, the derived macros recover its logical rules, coverage, equation induction, and falsity. The paired-formula composite is the internal equivalence

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big).$$

This is more than agreement on standard numerical examples.

3.2 The arithmetic interpretations

In one direction, interpret each fixed scalar iterator in iPRA by ordinary primitive recursion that ignores the recursion counter in its update. All the axioms of T_I follow.

For the reverse direction, the inherited internal argument has the following order, which must be respected to avoid a circular proof:

1. Construct a scalar predecessor P using equation induction and numerical cases, and prove $P0 = 0$, $P(Sn) = n$ throughout the original domain. No pairing or faithful equality test is assumed here.
2. Construct truncated subtraction and an equality characteristic; prove reflection of actual equality, not only correctness on standard numerals.
3. Translate every fixed quantifier-free formula into a characteristic equation. Obtain quantifier-free induction, including contextual instances.
4. With this induction available, construct pairing π and projections L, H, then compile every primitive-recursive description.

   For example, for the primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

the scalar compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})),\ u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the projection identities then prove both defining equations of f. This construction is external recursion on finite descriptions, not an internal evaluator of arbitrary program codes.

3.3 Both composites and consistency

An iterator is uniquely determined, internally, by its seed and update: equation induction on the equality of two candidate iterates proves this with all passive parameters fixed. The analogous argument holds for ordinary primitive recursion. External induction on finite descriptions therefore proves that every operation equals its double translation.

Both domains are the whole numerical domain, equality is actual equality, and the numerical isomorphism in either composite is the identity graph $x = y$. Structural induction on formulas, using the paired composite and the logical macros, proves both formula round trips. Contradiction is preserved by the explicit falsity bridge. Effective proof translations give equiconsistency. Neither theory is claimed to prove its own consistency.

These are inherited results, with their full written arithmetic development reproduced in Appendices A–E. The repository also provides the Lean statements `biInterp`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` [1]. No claim is made that the Python programs delivered here were verified by Lean, or that the upstream Lean project was rebuilt in this run.

4 The guarded public language of the smallest checker

The 520-byte and 921-byte programs have the same different interface: their public formulas are guarded Eq/D formulas, while auxiliary formulas used during a proof may be more general. This section gives the exact distinction needed for their bi-interpretation claims.

4.1 Public and auxiliary syntax

An equation is public. A paired formula $\mathsf{D}_{x,y}B$ is public if B is a compound public formula, or if its atomic matrix has the literal form

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{1}$$

The update terms may contain all the indicated variables and fixed parameters. Let this public language be $\mathcal{G}$.

Auxiliary proof formulas have the unrestricted syntax $(s = t)$, $\mathsf{Q}_y A$, and $\mathsf{D}_{x,y}A$. The primitive rules are Hyp, I0, IS, QE, DE, source-restricted Tr in auxiliary templates, and the single introduction

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases}$$

Freshness and discharge are as in Section 2. There is no atomic case choice. Every atomic public matrix has a provable zero instance by (1).

4.2 Sound interpretation of auxiliary formulas

Equality and Q retain their ordinary readings. For soundness of the entire auxiliary calculus use

$$(\mathsf{D}_{x,y}B)^\dagger = \begin{cases} \exists x \, \forall y \, (B(x, y) \to B(x, Sy)), & B \text{ is an equation,} \\ \exists x \, (B^\dagger(x, 0) \to \forall y B^\dagger(x, y)), & B \text{ is compound.} \end{cases}$$

Atomic introduction is sound immediately. Its elimination is sound because equation induction converts progress into the exported conditional closure. Compound introduction uses coverage, as before. All other rules have their ordinary sound interpretations.

On a guarded atomic matrix the zero instance is true. Consequently its progressive and conditional readings are equivalent: progress plus the true base gives all instances, and all instances imply progress. Structural induction gives agreement of the two readings throughout $\mathcal{G}$.

The qualification cannot be removed. For a matrix independent of x, take $E(y) \equiv (y = 1)$. In iPRA,

$$E(0) \to \forall y E(y)$$

is true because $0 \neq 1$, whereas

$$\forall y(E(y) \to E(Sy))$$

is false: the instance $y = 1$ would give $2 = 1$. Thus the two meanings already disagree on this very small unguarded matrix. Soundness of auxiliary calculations does not supply a formula round trip for every auxiliary expression.

4.3 Public completeness and the composites

For an equation E define

$$L(E) := \mathsf{D}_{r,z} G_z E,$$

where r is unused. This is public and interderivable with E: forward introduction ignores the progress assumption; backward elimination uses the true zero gate and its instance at 1. Let $P(A) = L(A)$ for an equation, and $P(A) = A$ for a compound. Then the translation from the old Eq/D language is

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} P(TB).$$

An old Q is represented by an unused existential coordinate.

The target matrix of every translated old case introduction is compound, so the deterministic case rule suffices. Old atomic progress transfers using auxiliary equation induction under the padded base, followed by compound introduction. Elimination pads and unpads its instances. Equality is recovered using the inherited zero-source admissibility proof; public-template transport follows by induction on public syntax. At guarded atomic binders the zero base is a theorem, which is precisely what is needed in that induction.

Writing J for literal inclusion of public formulas into the old native language, the padding equivalence and binder congruence give

$$A \dashv\vdash JT(A), \qquad B \dashv\vdash TJ(B) \quad (B \in \mathcal{G}).$$

Together with auxiliary soundness and agreement on public formulas, these imply equality of the public consequence relations. Composing with Section 3 gives the claimed whole-domain bi-interpretability and equiconsistency of the public theory. This argument is inherited from the deterministic-auxiliary construction [4]; its complete source is included among the references. It is unchanged by the implementations below.

5 Removing duplicate validation in the guarded checker

This is the main new implementation result. All kernel instructions, context operations, scope updates, and input/output behavior are kept. Only the public-goal validator is simplified.

5.1 The representation and the original predicate

The public codec uses arrays of arity one for D, two for equality, one for numerical successor, and three for iteration. Protected numerical slots are numbered with canonical representatives $3, 5, 7, \ldots$; zero is the integer 0. The extra slots under successor are dummy zero slots in the inherited codec. They are not additional object-language binders. Raw negative labels are allocated rigid parameters inside proofs, not canonical free names in a closed public goal.

If p is the number of protected slots, the original term validator accepts an exact Python integer precisely when

$$a = 0 \quad \text{or} \quad 3 \le a < 3 + 2p \text{ and } a \text{ is odd.}$$

It accepts exact lists of arities one and three, recursing with $p + 2$ and $p + 1$, respectively. The formula validator accepts Eq and D; Eq passes the same p to its terms and D passes $p + 2$ to its matrix. If that matrix is atomic, each side must be a ternary iterator with literal first two entries $[5, 0]$. This is exactly (1) in the protected-slot codec.

5.2 The smaller shape predicate

The new predicate retains only a Boolean sort d: false for formula, true for term. It checks exact types and legal arities, and checks that each numerical integer is either zero or odd and at least three. After recursive shape validation, an atomic matrix under D is checked by the two-entry prefix test alone:

```
all(b[:2] == [5, 0] for b in a[0])
```

No separate iterator-type or arity test is necessary at this point.

Lemma 5.1 (Guard prefix after term validation)**.** *On already shape-validated matrix sides, the new prefix check has exactly the success cases of the old explicit guard check, with exceptions counted as rejection.*

Proof. A side is either an exact integer or an exact list with arity one or three. Slicing an integer raises and rejects. A unary list has a prefix of length one and cannot equal $[5, 0]$. For a ternary list, equality of the prefix states exactly the old two coordinate tests. Its update has already been recursively validated. These exhaust the possibilities. □

5.3 The existing opener supplies every omitted bound

The kernel already begins each call with a no-image opening of the goal. For an array of arity r, its traversal changes numerical depth from e to $e + 5 - r$. A formula starts at $e = 0$. Equality adds three; D and successor add four; iteration adds two. Thus term positions have odd depth and formula positions have even depth.

At initial name floor $k = 0$, a scalar is left unchanged exactly when $0 \le a < e$. An out-of-range term leaf tries to insert the absent image. In the byte-profile implementation that default image is the function object `c`; the scalar arithmetic operation on it raises. This failure happens before a certificate instruction is read.

Lemma 5.2 (Shape and initial opening)**.** *Let V be the old guarded public-goal validator, W the new shape predicate, and O successful initial no-image opening. For every finite JSON goal, abstracting from implementation resource limits,*

$$V(A) \quad \Longleftrightarrow \quad W(A) \wedge O(A).$$

Whenever either side holds, the opener leaves A literally unchanged.

Proof. At a numerical position with p protected slots, the traversal depth is $3 + 2p$. The new shape check supplies exact integer type, nonnegativity and the canonical parity/lower-bound conditions. Successful opening supplies exactly the remaining upper bound $a < 3 + 2p$. All admitted scalar leaves are numerical, so their depths are odd; an invalid leaf cannot be discarded as a true formula annotation. Every public constructor is recursively visited. In-range leaves and array structure are unchanged. The preceding lemma supplies exactly the old atomic guard. Induction on finite input shape proves the result. □

Theorem 5.3 (Preservation of the guarded certificate relation)**.** *The 921-byte guarded module and its 1,201-byte comparator define the same accepted mathematical certificate relation, abstracting from implementation resource limits. In particular they accept exactly the same proofs of every canonical guarded public formula.*

Proof. The parser, goal extraction, kernel and final success marker are identical. If the old validator accepts, the new one accepts and the initial opener sees the same goal and succeeds. All subsequent instructions are identical. If the new predicate rejects, the old one cannot accept. If the new shape predicate succeeds but the old predicate fails, the initial opener fails before any instruction is consumed. Thus no malformed proof suffix or inert prefix can make that case accept. This proves both directions. □

Runtime memory and recursion limits are excluded from this mathematical identity. A validator refactoring can change the stack depth at which an extremely nested input raises `RecursionError`. The finite process comparisons record actual behavior within the tested bounds. The theorem does not assert identical resource exhaustion thresholds.

6 The two smaller local changes

6.1 Full native public formulas: 803 to 788 bytes

The native Eq/D public interface does not impose the guard (1). Its kernel retains the inherited introduction-mode choice. Its validator already factors scope checking as described in the incoming `one_context` package. The present change merely replaces its early integer return and remaining list return by one conditional expression:

```
return integer_result if type(a) is int else list_result
```

Both branch expressions, their short-circuit order, and recursive calls are unchanged. On an exact integer both versions evaluate only the integer expression; otherwise both evaluate only the same list expression. Structural induction gives the same predicate on finite inputs. The rest of the module has identical AST. Consequently the accepted certificate relation and inherited full-native metatheory are unchanged. This saves 15 normalized bytes, one token and one AST node. It is a code-structure improvement, not an axiom deletion.

6.2 Compact guarded profile: 521 to 520 bytes

The entire change is

```
([s, u] if s else [t] * 3)
```

replaced by

```
(s and [s, u] or [t] * 3)
```

If s is false, both choose the triplicated template. If s is true, the list $[s, u]$ is nonempty and therefore true, so both choose it. The branch values and relevant evaluations agree for all parsed JSON values, not merely canonical mathematical inputs. Nothing else changes.

This reduction worsens AST size from 300 to 303 and gzip size from 327 to 329. It is reported because it gives a strictly smaller normalized byte coordinate under the repository metric, but its mathematical significance should not be exaggerated.

7 Rejected shortcuts and remaining mathematical work

7.1 Why context length did not remove freshness cost

For the local-scope checker, the exact entry invariant is

$$k = 1 - |g|.$$

It holds initially, major-premise recursion preserves the same context, and each continuation appends one resource while decrementing k. Thus a two-argument implementation can reconstruct k at entry. The correct version, however, measures 529 normalized bytes.

An apparently 514-byte version reconstructed a fixed threshold from context length and removed the decrement before final opening. This is invalid: the opener closes over k, and the original program updates that bound *before* validating the inserted fresh name. The altered version rejects valid proofs. It is not delivered as a smaller kernel. The correct 529-byte variant and analysis are retained only as a research note, not a new best profile.

7.2 Other non-results

Deleting public Q syntax would repeat inherited clause syntax. One-sided crossed transport would repeat the current incoming proposal. Removing image revalidation during substitution was not justified: its role includes rejecting out-of-scope witness terms before a fresh eigenparameter becomes available. Introducing an arbitrary quantified matrix into the progress rule would strengthen the theory toward full formula induction and is not an allowed simplification of iPRA.

No smaller initialized-iteration basis was established. In particular, introducing a function by a total, unique quantifier-free graph does not by itself show that an equation-induction or quantifier-free-induction schema is preserved in the expanded language. Eliminating the new function can introduce existential quantifiers. A proposed zero-start iterator bootstrap must discharge this obligation, rather than borrowing the existing scalar bridge unchanged.

7.3 Further research questions

1. Can the case/progress interface be simplified on the full native language without changing the meaning of unguarded atomic D formulas? A solution should explain both formula composites, including the $E(y) \equiv y = 1$ test in Section 4.

2. Can administrative Q resources be made unforgeable by the certificate format, so that one simpler elimination or introduction dispatcher suffices? The format decoder and its scope checks must be included in the measured trusted module.

3. Can a genuinely new scalar predecessor construction avoid one term constructor or one computation schema? Any proposal must work in arbitrary models, preserve restricted induction, and establish the internal uniqueness needed for the two interpretation composites.

4. Can a finite syntax-directed validator be mechanically shown to refine the existing Lean native kernel? That would close the remaining gap between the formalized mathematics and the Python certificate codec.

5. Can one program improve all three coordinates of the current compact envelope? The 520-byte profile does not do this, and gzip size should continue to be reported separately rather than optimized.

8 Artifacts, reproduction, and assurance

The archive contains the three complete standalone programs, exact comparators, source provenance, representative proof and rejection certificates, machine-readable claims, measurements, validation receipts, this LaTeX source and its compiled PDF. No checker imports a producer, measurement module, or test helper.

Run the portable reproduction with Python 3.12:

```
python3 reproduce.py
```

The normalization convention is

```
ast.unparse(ast.parse(source)) + '\n'
```

The complete normalized UTF-8 source is measured. AST size counts every node from `ast.walk`. Significant tokens exclude encoding, comments, whitespace/indentation, newline markers and end markers. Gzip uses compression level 9 and timestamp zero. The run used CPython 3.12.14. Exact environment details and source hashes are in the measurement receipt.

During development the guarded refinement passed 30,255 validator-factorization comparisons and 284 full process runs on 71 fixtures, across normal and optimized Python. The native refinement passed 10,255 validator comparisons and 168 full process runs on 42 fixtures. There were no mismatches or timeouts. Some inherited inputs are expected rejections because they belong to another public interface; the receipts record actual outcomes rather than treating every supplied file as a positive theorem. The portable reproduction is a separate selected suite whose exact count is recorded in its own receipt.

These finite checks support the implementations. They do not replace the universal preservation proofs in Sections 5 and 6, nor prove the inherited mathematical bridge by testing. The new metatheorems were also independently reviewed within this run. There was no external human peer review and no new Lean or Rocq verification. The archive makes no claim of minimality or of a new mathematical axiomatization.

Attribution and scope of the inherited appendices

The following five sections are reproduced, without changes to their source text, from the foundation files included in the pinned incoming `kernel_refinement` package. They are existing work in ProveIt, not new results of this investigation. Their original labels, theorem titles, and occasional references to "this paper" or "new" are retained as part of the attributed source. The present report's new claims are exactly those identified in Sections 5 and 6. The remaining foundation files, the guarded composition proof and the codec proof are supplied as source artifacts in the archive and are listed in the provenance map.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x (B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [5] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

B.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{2}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{3}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

B.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{4}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{5}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{6}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (6) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{7}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (6); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

B.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{8}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (7), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{9}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{10}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

B.4 Saturated iteration gives predecessor

We now need only the three equations (10). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (11) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (12) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (13)$$

The base is (12). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (14)$$

The first uses the comparator recurrence. The second uses (13) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (15)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{16}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (15) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

B.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

C From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [6]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix B, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

C.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion.

In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{17}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{18}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{19}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{20}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{21}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (21) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

C.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{22}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{23}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{24}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (17). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (23); symmetry and (20) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{25}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{26}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{27}$$

To prove the range assertion, split (19). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (18) also give, for all u, v,

$$C(u, v, v) = v. \tag{28}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{29}$$

If $u = 1 = S0$, (24) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

C.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{30}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{31}$$

Using these equations and (21), we now derive

$$0 + y = y, \tag{32}$$

$$(Sx) + y = S(x + y), \tag{33}$$

$$d(Sx, Sy) = d(x, y), \tag{34}$$

$$d(Sa, a) = 1. \tag{35}$$

Each induction below has a single equation as its induction formula.

For (32), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (33), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (33) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{36}$$

Indeed, both sides are $S(y + r)$.

For (34), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (35). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (34) and the induction hypothesis.

C.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{37}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (32) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (28).

For the step, assume (37) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (37) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (36) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (37). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{38}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (37). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (35) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{39}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{40}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{41}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (26), which is equivalent to $d(Sa, b) = 1$ by (29), and then to $a = b$ by (38). The range assertion is (25) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (40) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{42}$$

is a theorem, not an extra hypothesis.

C.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{43}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{44}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{45}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{46}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{47}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (40)–(41). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (47) for every quantifier-free formula.

C.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (47), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

D Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [7]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

D.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

D.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

D.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

E Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u,\vec{p})$, introduce $i_h(n,a,\vec{p})$ with the two equations

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

E.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem E.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, pinned snapshot `8eb71c1e570e07ef45ead686628e77f18ef34798`, https://github.com/VladimirReshetnikov/ProveIt/tree/8eb71c1e570e07ef45ead686628e77f18ef34798/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] The same snapshot, `docs/incoming/`: `ipra_compact_update_2026-11-11.zip`, `ipra_kernel_refinement_2026-11-11.zip`, and `ipra_one_context_2026-11-11.zip`. Exact Git blob hashes, source hashes and extracted locations are supplied in the archive provenance.

[3] ProveIt, inherited Choice-DI basis and ordinary bridge, `Research/certified_zero/reference/inherited_choice_basis.tex` and `inherited_ordinary_bridge.tex`, also reproduced in the pinned incoming foundation files. See the attributed appendices above.

[4] ProveIt, *A deterministic calculus with auxiliary formulas*, `Research/deterministic_auxiliary/theory/composition.tex`, at the snapshot in [1]. This is the source of the guarded-public consequence theorem and the auxiliary/public distinction.

[5] Inherited scalar predecessor development, as attributed in `reference/foundations/scalar_predecessor.tex`; copied from the foundation chain of the pinned incoming package [2].

[6] Inherited forward-left equality-reflection development, as attributed in `reference/foundations/equality_reflection.tex`; complete source and proof reproduced in this report's appendix.

[7] Inherited primitive-recursion compiler and pairing development, `reference/foundations/primitive_recursion.tex`; complete source copied from the pinned incoming package [2].